

Reporte Técnico

Práctica ETL con Python, SQLite3 y Expresiones Regulares

Archivo fuente	members.csv (5 000 registros)
Base de datos	members.db (SQLite3)
Script principal	data_practica.py
Librerías usadas	pandas · sqlite3 · re · datetime

1. Creación del Ambiente Virtual

Un **ambiente virtual** es un entorno Python aislado que permite instalar dependencias específicas sin afectar la instalación global del sistema. Es una práctica recomendada en proyectos de *machine learning* y ciencia de datos.

Crear el entorno:

```
python -m venv venv_mlearning
```

Activar (Linux/Mac):

```
source venv_mlearning/bin/activate
```

Activar (Windows):

```
venv_mlearning\Scripts\activate
```

Instalar dependencias:

```
pip install pandas
```

Desactivar al terminar:

```
deactivate
```

2. Carga del CSV en SQLite3

SQLite3 es un motor de base de datos relacional embebido incluido en la biblioteca estándar de Python. No requiere servidor ni configuración adicional; almacena toda la información en un único archivo *.db*.

Código utilizado:

```
conn = sqlite3.connect('members.db') df.to_sql('members', conn, if_exists='replace', index=False)
```

Se crearon dos tablas: **members** (datos originales) y **members_transformed** (datos procesados).

3. ETL — Extracción del Código Postal (str.extract)

La función **extract_postal_code()** usa **str.extract()** de pandas, que aplica una Expresión Regular sobre cada elemento de la Serie y devuelve el grupo de captura como nueva columna.

Patrón RegEx:

```
pattern = r'\b(\d{5})\b'
```

Componente	Significado
\b	Límite de palabra (word boundary)
\d{5}	Exactamente 5 dígitos consecutivos
()	Grupo de captura que devuelve str.extract()

Ejemplo:

Dirección original	postal_code
Gg. Monginsidi No. 08 - Medan, Aceh 80734	80734
Gg. Rajawali Timur No. 7 - Prabumulih, MA 09434	09434
Jalan Kebonjati No. 0 - Ambon, SS 57739	57739

Nulos encontrados: **0 de 5 000** ✓

4. ETL — Limpieza del Número Telefónico (str.replace)

La función **clean_phone_number()** emplea **str.replace()** con una Expresión Regular para eliminar todos los caracteres no numéricos del campo *phone_number*, dejando únicamente dígitos.

Patrón RegEx:

```
pattern = r'[+()\-\\s]'
```

Carácter	Descripción
+	Símbolo de prefijo internacional
()	Paréntesis de apertura y cierre
-	Guion separador
\\s	Espacios en blanco (space, tab, etc.)

Ejemplo:

phone_number original	phone_clean
(0151) 081 2706	01510812706
+62 (036) 461 7027	620364617027
+62 (418) 329-4756	624183294756
089 833 6695	0898336695

5. ETL — Formato de Fecha de Nacimiento (RegEx)

La columna *birth_date* tiene dos formatos distintos. La función **format_birth_date()** los normaliza a **DD/MM/YYYY** con la siguiente Expresión Regular:

```
pattern = re.compile(r'(\d{1,2})[\s\-]([a-zA-Z]{3})[\s\-](\d{2,4})', re.IGNORECASE)
```

Grupo RegEx	Captura	Ejemplo
(\d{1,2})	Día (1 ó 2 dígitos)	05 / 11
[\s-]	Separador (espacio o guión)	- ó ''
([a-zA-Z]{3})	Nombre de mes (3 letras)	feb / Jan
[\s-]	Separador	- ó ''
(\d{2,4})	Año (2 ó 4 dígitos)	91 / 1993

Los años de 2 dígitos se expanden: si el valor es > 25 se antepone '19'; si es ≤ 25 se antepone '20'.

Ejemplo:

birth_date original	birth_date_fmt
05-feb-91	05/02/1991
11 Jan 1993	11/01/1993
31-jul-93	31/07/1993
22 Jan 2003	22/01/2003

6. ETL — Cálculo de Edad desde register_time

La función **calculate_age()** combina la fecha de nacimiento normalizada con el campo *register_time* (Unix timestamp) para calcular la edad exacta en años al momento del registro.

Algoritmo paso a paso:

1. Convertir *register_time* (int Unix) a **datetime** con *datetime.fromtimestamp()*.
2. Parsear *birth_date_fmt* con *datetime.strptime(v, '%d/%m/%Y')*.
3. Calcular diferencia en días: *(ref_date - born).days*.
4. Dividir entre **365.25** (contempla años bisiestos) y truncar con *int()*.

birth_date_fmt	register_time	age_at_register
05/02/1991	1617634046	30
11/01/1993	1617634018	28
31/07/1993	1617634035	27
04/11/1991	1617634034	29
22/01/2003	1617634010	18

7. Vista Previa del Dataset Transformado

Primeras 6 filas de la tabla **members_transformed** en SQLite3 con todas las columnas derivadas:

first_name	last_name	postal_code	phone_clean	birth_date_fmt	age_at_register
Citra	Nurdiyanti	80734	01510812706	05/02/1991	30
Aris	Setiawan	09434	620364617027	11/01/1993	28
Cornelia	Handayani	57739	0898336695	31/07/1993	27
Soleh	Rajasa	76156	624183294756	04/11/1991	29
Vivi	Astuti	70074	812511835	22/01/2003	18
Bahuraksa	Mangunsong	00880	62113697082	13/11/1997	23

8. Resumen de Técnicas Utilizadas

Función	Método pandas	Patrón RegEx	Propósito
extract_postal_code()	str.extract()	\b(\d{5})\b	Extraer código postal
clean_phone_number()	str.replace()	[+()\-s]	Eliminar símbolos del teléfono
format_birth_date()	series.apply()	(\d{1,2})([s-]([a-zA-Z]{3})[s-])(\d{2,4})	Normalizar fecha a DD/MM/YYYY
calculate_age()	DataFrame.apply()	N/A (datetime)	Calcular edad desde Unix timestamp